

HRMS Authentication & Authorization Architecture

Compact Architecture Reference — JWT, Current User, Roles, Permissions and Policies

1. Technology & Architecture

The HRMS uses .NET 9, Clean Architecture, CQRS + MediatR, EF Core, SQL Server, Repository Pattern, FluentValidation, AutoMapper, Serilog, JWT Authentication, and Role + Permission Authorization.

```
Projects
HRMS.API
HRMS.Application
HRMS.Domain
HRMS.Infrastructure
HRMS.Persistence
HRMS.Shared
```

2. Authentication Architecture

User and Employee are separate concepts. UserId identifies the authentication account; EmployeeId identifies the HR/business employee record.

```
User
    ■■■ EmployeeId
```

UserId → Authentication, Authorization, Audit, Logging
EmployeeId → Employee profile, Bank details, Attendance, Leave, etc.

3. JWT

The application has IJwtTokenService, JwtTokenService and JwtOptions. The JWT contains identity claims such as sub, userId, employeeId, name, jti, role, exp, iss and aud. ASP.NET Core validates the signature, issuer, audience and expiration.

```
JWT
    ■■■ sub
    ■■■ userId
    ■■■ employeeId
    ■■■ name
    ■■■ jti
    ■■■ role
    ■■■ exp
    ■■■ iss
    ■■■ aud
```

4. Current User Context

```
public interface IUserContext
{
    Guid? UserId { get; }
    Guid? EmployeeId { get; }
    string? UserName { get; }
    bool IsAuthenticated { get; }
}
```

CurrentUserContext reads claims from HttpContext.User. AddHttpContextAccessor() makes IHttpContextAccessor available so the current request and its claims can be accessed.

```
builder.Services.AddHttpContextAccessor();

builder.Services.AddScoped<CurrentUserContext>();
builder.Services.AddScoped<IUserContext>({
    sp => sp.GetRequiredService<CurrentUserContext>());
```

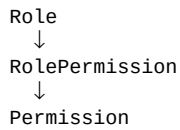
5. Roles

A user can have one or multiple roles through the UserRole junction entity.

```
User
    ■■■ UserRole
        ■■■ Role
```

6. Permissions

Permission represents a specific capability. RolePermission is the many-to-many bridge between Role and Permission.



Initial permissions:

```
Employee.View
Employee.Create
Employee.Update
Employee.Delete
```

7. Current Role-Permission Mapping

Role	Permissions
Admin	Employee.View, Employee.Create, Employee.Update, Employee.Delete
HR	Employee.View, Employee.Create, Employee.Update
Manager	Employee.View
Employee	Employee.View
TestRole	None (used to verify denial)

8. EF Core / Database

ApplicationDbContext contains DbSet and DbSet. PermissionConfiguration defines the Permissions table and unique permission name. RolePermissionConfiguration defines a composite primary key of RoleId + PermissionId and both foreign keys. The migration and database update have been completed.

```
Tables
Users
UserRoles
Roles
RolePermissions
Permissions
```

```
RolePermissions primary key:
RoleId + PermissionId
```

9. Permission-Based Authorization

The final HRMS design does not rely on [Authorize(Roles = "HR")] throughout controllers. Role answers 'who is the user?' while permission answers 'what can the user do?'.

```
[Authorize(Policy = "Employee.Update")]
```

10. Policy

A policy is a named authorization rule. For example, the Employee.Update policy says that the request must satisfy the Employee.Update authorization requirement.

```
Policy
Name: Employee.Update
Requirement: PermissionRequirement("Employee.Update")
```

11. Custom Authorization Requirement

Custom means we create our own requirement class for the HRMS permission model. ASP.NET Core provides the authorization framework, but it does not know the HRMS User → Role → RolePermission → Permission relationship.

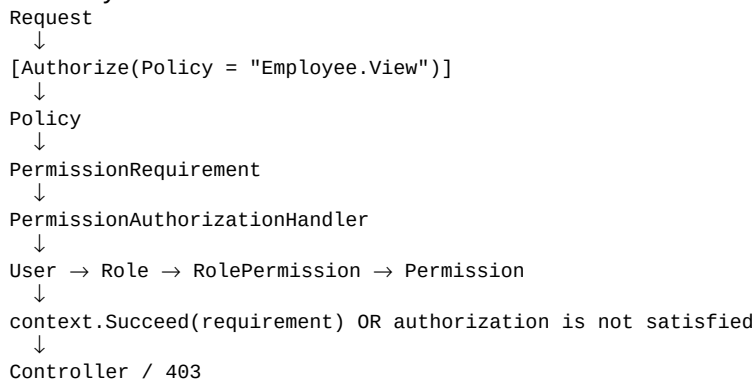
```
public class PermissionRequirement : IAuthorizationRequirement
{
    public string Permission { get; }

    public PermissionRequirement(string permission)
    {
        Permission = permission;
    }
}
```

```
}
```

12. Permission Authorization Handler

PermissionAuthorizationHandler evaluates PermissionRequirement. It is registered with dependency injection and is called automatically by ASP.NET Core when an endpoint uses a matching policy. We never call the handler manually.



13. DI Registration

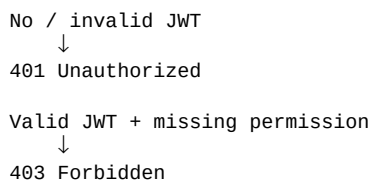
```
services.AddScoped<IAuthorizationHandler,  
    PermissionAuthorizationHandler>();
```

Authorization policies are kept in a separate AuthorizationConfiguration.cs rather than putting all policy definitions directly into Program.cs. Infrastructure registers the authorization setup through AddInfrastructure().

14. Controller Usage

```
[HttpGet]  
[Authorize(Policy = "Employee.View")]  
public async Task<IActionResult> GetEmployees(...)  
  
[HttpPost]  
[Authorize(Policy = "Employee.Create")]  
public async Task<IActionResult> CreateEmployee(...)  
  
[HttpPut("{id}")]  
[Authorize(Policy = "Employee.Update")]  
public async Task<IActionResult> UpdateEmployee(...)  
  
[HttpDelete("{id}")]  
[Authorize(Policy = "Employee.Delete")]  
public async Task<IActionResult> DeleteEmployee(...)
```

15. Authentication vs Authorization



16. Testing Completed

A user with the Employee role and Employee.View permission successfully accessed the protected Employee GET endpoint with 200 OK. A test user assigned to TestRole with no permissions received 403 Forbidden when calling the same endpoint. This proves that the permission policy and authorization handler are actually enforcing database permissions.

17. Adding a New Permission — Example Employee.Download

Current implementation requires four changes: add the permission record, assign it to roles, register its policy, and protect the endpoint. No changes are needed to PermissionRequirement or PermissionAuthorizationHandler.

1. Permissions table

```

Employee.Download

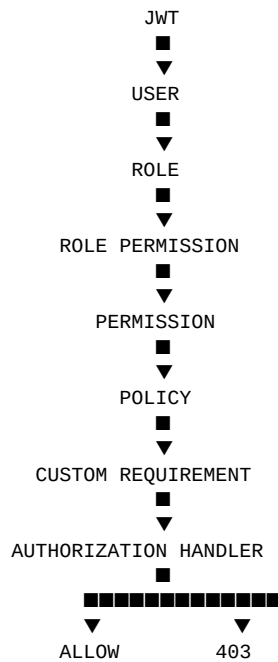
2. RolePermissions
HR → Employee.Download
Admin → Employee.Download

3. AuthorizationConfiguration
options.AddPolicy("Employee.Download", policy =>
{
    policy.Requirements.Add(
        new PermissionRequirement("Employee.Download"));
});

4. Controller
[Authorize(Policy = "Employee.Download")]

```

18. Final Architecture



19. Current Status

Area	Status
Employee CRUD	Complete
User / Employee relationship	Complete
Login / JWT	Complete
CurrentUserContext	Complete
Roles / UserRole	Complete
Permissions / RolePermission	Complete
Policies / Requirement	Complete
Authorization Handler	Complete
Permission testing	Complete
Permission Authorization Foundation	COMPLETE

Next logical step: continue building the next HRMS business module while reusing the established authentication, role, permission and policy foundation.